



UMSS

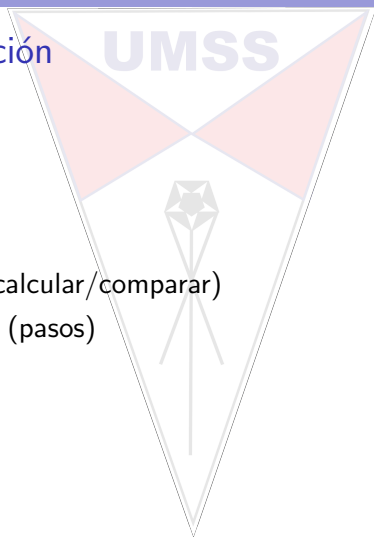
Correctitud
Eficiencia

e presenta debe ser eficiente.
preguntas son:
n?
edir la lógica de pensamiento?

La solución que se presenta debe ser eficiente.
Por lo que, las preguntas son:

- ▶ Cuánto tardan?
- ▶ Es posible medir la lógica de pensamiento?

- ▶ Expresiones (calcular/comparar)
- ▶ Instrucciones (pasos)



$$E = \begin{cases} \text{var} & \Rightarrow T_E(n) = 1 \\ \text{ctte} & \Rightarrow T_E(n) = 1 \\ E_1 \text{ op } E_2 & \Rightarrow T_E(n) = T_{E_1}(n) + T_{E_2}(n) + 1 \\ \text{op } E_1 & \Rightarrow T_E(n) = T_{E_1}(n) + 1 \\ (E_1) & \Rightarrow T_E(n) = T_{E_1}(n) \\ \text{fn}() & \Rightarrow T_E(n) = T_{\text{fn}()}(n) \end{cases}$$

Definición

Tiempo de ejecución - Instrucciones

Escritura: **mostrar E**

$$T_{out}(n) \equiv T_E(n)$$

Definición

Tiempo de ejecución - Instrucciones

Retorno: **return E**

$$T_{ret}(n) \equiv T_E(n)$$

Tiempo de ejecución - Instrucciones

Asignación: **var = E**

$$T_{asig}(n) \equiv T_E(n) + 1 \equiv T_E(n)$$

Tiempo de ejecución - Instrucciones

Condicional: **if_then_else**

```
1  if (expBool) {  
2      cuerpoSI  
3  } else {  
4      cuerpoNO  
5  }
```

$$T_{if}(n) \equiv T_{expBool}(n) + \max(T_{cuerpoSI}(n), T_{cuerpoNO}(n))$$

Definición

Tiempo de ejecución - Instrucciones

Repetición por conteo: **for**

```

1  for(cont = valorIni; cont <= valorFin; cont+=k){
2      cuerpo
3  }
```

 $\text{valorIni} \leq \text{cont} \leq \text{valorFin}; +k$

$$T_{for}(n) \equiv \frac{\text{valorFin} - \text{valorIni}}{k} + 2 + \left(\frac{\text{valorFin} - \text{valorIni}}{k} + 1 \right) * T_{cuerpo}(n)$$

Tiempo de ejecución - Instrucciones

Repetición por condición: **while_do**

```
1  while (expBool) {  
2      cuerpo  
3  }
```

$m \equiv$ numero de veces que la $expBool$ se hace verdadera

$$T_{while}(n) \equiv (m + 1) * T_{expBool}(n) + m * T_{cuerpo}(n)$$

Tiempo de ejecución - Instrucciones

Repetición por condición: **do_while**

```
1  do{  
2      cuerpo  
3  }while(expBool);
```

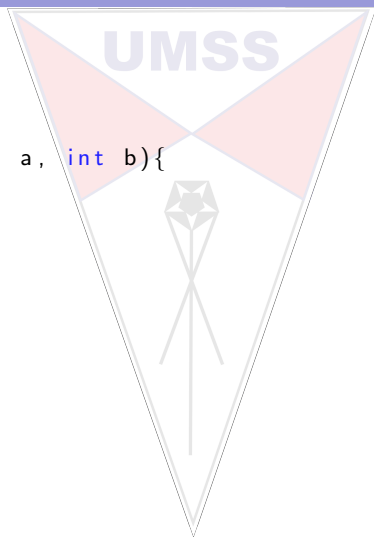
$m \equiv$ numero de veces que la $expBool$ se hace verdadera

$$T_{do}(n) \equiv (m + 1) * (T_{expBool}(n) + T_{cuerpo}(n))$$

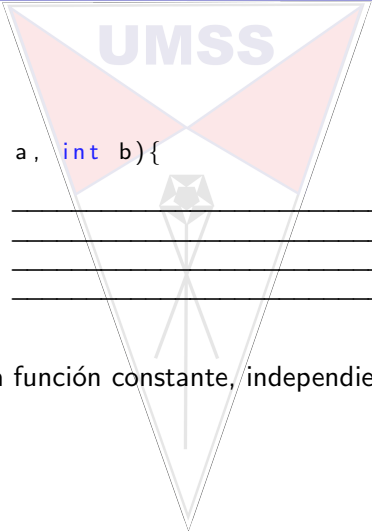
Ejemplos

```
1 int mayor(int a, int b){  
2   int c, d;  
3   c = |a-b|;  
4   d = a+b;  
5   d = (c+d)/2;  
6   return d;  
7 }
```

Cuál es el $T(n)$?



Ejemplos



```
1  int mayor(int a, int b){
2      int c, d;
3      c = |a-b|;
4      d = a+b;
5      d = (c+d)/2;
6      return d;
7  }
```

_____	4
_____	3
_____	5
_____	1
=====	13

$T(n) \equiv 13$; es una función constante, independiente de n

Ejemplos

```

1  int sumDig(int n){
2      int s;
3      s = 0;
4      while(n > 0){
5          s = s + n%10;
6          n = n/10;
7      }
8      return s;
9  }

```

$$\begin{array}{rcll} & & (m+1)*3 & 1 \\ \hline 5 & | & + & | \\ 3 & | & 8*m & | \\ \hline & & 11m+3 & 1 \\ \hline & & 11m+5 & \end{array}$$

$T(n) \equiv 11m + 5$; pero que es m ?

Ejemplos

```
1 int sumardig(int n){
2     int s;
3     s = 0;
4     while(n > 0){
5         s = s + n%10;
6         n = n/10;
7     }
8     return s;
9 }
```

$$\begin{array}{r}
 \hline
 (m+1) \cdot 3 \\
 \hline
 \begin{array}{r}
 5 \\
 3
 \end{array} \mid \begin{array}{r} \\ 8 * m
 \end{array} \mid \begin{array}{r} + \\ 8 * m
 \end{array} \\
 \hline
 \hline
 11m + 3 \\
 \hline
 1 \\
 \hline
 \hline
 11m + 5
 \end{array}$$

$$T(n) \equiv 11m + 5$$

$$T(n) \equiv 11 * (\log_{10} n + 1) + 5$$

$$T(n) \equiv 11 * \log_{10} n + 16$$

$T(n)$ es una función logarítmica, dependiente de n

```
int n){
{
%10;      _____ 5 |
          _____ 3 | 8 * m
          _____
          _____
          _____
```

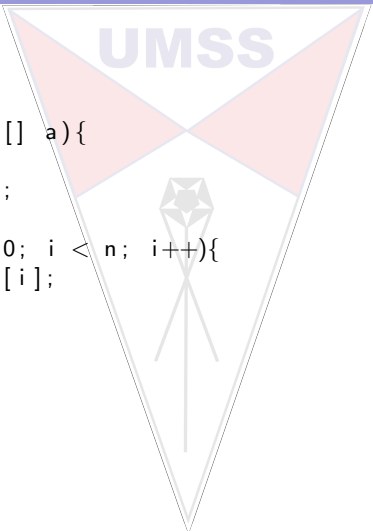
$n + 1) + 5$

$n + 16$

ón logarítmica, dependiente de

$$T(n) = \begin{cases} 11 * \log_{10} n + 16 & \Leftrightarrow n > 0 \\ 5 & \Leftrightarrow \text{caso contrario} \end{cases}$$

Ejemplos



```
1 int sumar(int[] a){  
2     int n, s;  
3     n = a.length;  
4     s = 0;  
5     for(int i = 0; i < n; i++){  
6         s = s + a[i];  
7     }  
8     return s;  
9 }
```

Cuál es el $T(n)$?

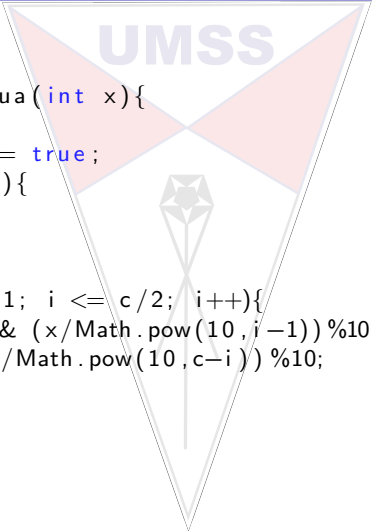
Ejemplos

```
1 boolean sumar(int [] a){
2   int n, s;
3   n = a.length;
4   s = 0;
5   for(int i = 0; i <= n-1; i++){
6       s = s + a[i];
7   }
8   return s;
9 }
```

				1
				1
		$n-1-0+2$	$n+1$	
3	*	n	$3n$	
			$4n+1$	
				1
				$4 + 4n$

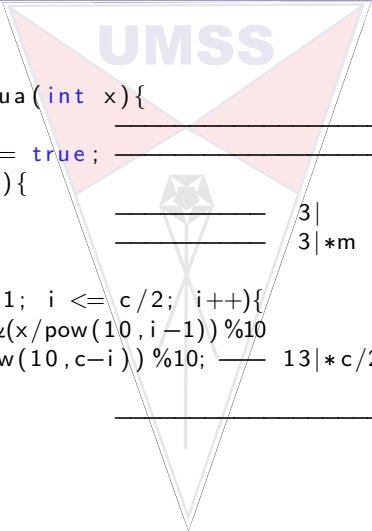
$T(n) \equiv 4 + 4n$; es una función lineal, es dependiente del tamaño de entrada

Ejemplos



```
1 boolean capicua(int x){
2     int c = 1;
3     boolean res = true;
4     while( x > 9){
5         c++;
6         x = x/10;
7     }
8     for(int i = 1; i <= c/2; i++){
9         res = res && (x/Math.pow(10,i-1))%10
10            == (x/Math.pow(10,c-i))%10;
11     }
12     return res;
13 }
```

Ejemplos



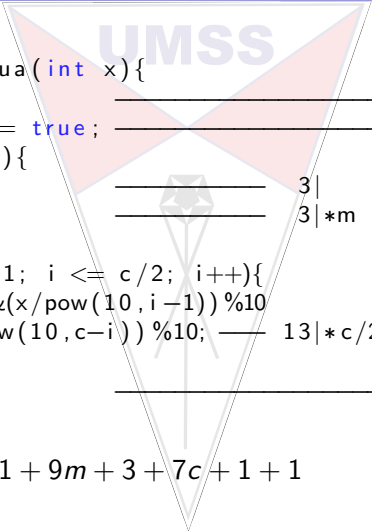
```

1  boolean capicua(int x){
2      int c = 1;
3      boolean res = true;
4      while( x > 9){
5          c++;
6          x = x/10;
7      }
8      for(int i = 1; i <= c/2; i++){
9          res = res&&(x/pow(10,i-1))%10
10             == (x/pow(10,c-i))%10;
11      }
12      return res;
13  }
```

_____	1
_____	1

_____	$3 \mid (m+1)*3 \mid$
_____	$3 \mid + \mid$
_____	$3 \mid *m \mid 6m \mid$
_____	$9m+3$
_____	$c/2-1+2 \mid$
_____	$+ \mid$
_____	$13 \mid *c/2-1+1 \mid 13c/2 \mid$
_____	$7c+1$
_____	1

Ejemplos



```

1  boolean capicua(int x){
2      int c = 1;
3      boolean res = true;
4      while( x > 9){
5          c++;
6          x = x/10;
7      }
8      for(int i = 1; i <= c/2; i++){
9          res = res&&(x/pow(10,i-1))%10
10             == (x/pow(10,c-i))%10;
11      }
12      return res;
13  }
```

	1
	1
	$(m+1)*3$
	$+$
$3 \mid 3 \mid m$	$6m$
	$9m+3$
	$c/2-1+2$
	$+$
$13 \mid *c/2-1+1$	$13c/2$
	$7c+1$
	1

$$T_{capicua}(x) \equiv 1 + 1 + 9m + 3 + 7c + 1 + 1$$

$$\equiv 7 + 9m + 7c$$

Ejemplos

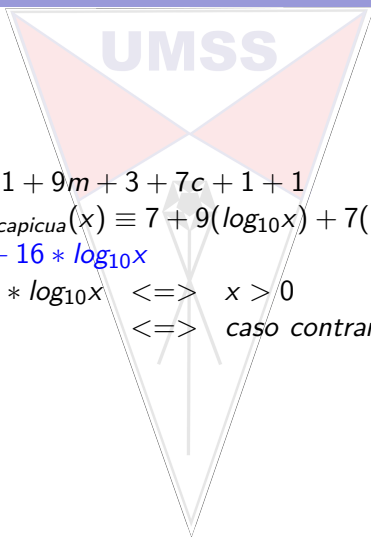
Cuánto es m? Cuánto es c?

```
1  int c = 1;
2  while( x > 9){
3      c++;
4      x = x/10;
5  }
```

x	$x > 9$	nro veces True	c	
x	True	1	2	$x/10^{m-1} = 10$
$x/10^1$	True	2	3	$x = 10 * 10^{m-1}$
$x/10^2$	True	3	4	$x = 10^m$
...	...	...	...	$\log_{10} x = m$
$x/10^{m-1}$	True	m	m+1	
$x/10^m$	False	m+1		

$$m = \log_{10} x; c = \log_{10} x + 1$$

Ejemplos



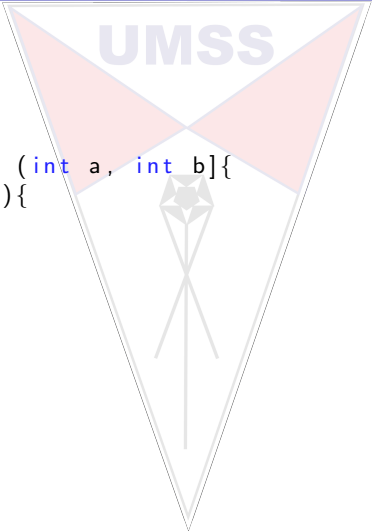
$$T_{capicua}(x) \equiv 1 + 1 + 9m + 3 + 7c + 1 + 1$$

$$\equiv 7 + 9m + 7c \quad T_{capicua}(x) \equiv 7 + 9(\log_{10}x) + 7(\log_{10}x + 1)$$

$$T_{capicua}(x) \equiv 14 + 16 * \log_{10}x$$

$$T(n) = \begin{cases} 14 + 16 * \log_{10}x & \Leftrightarrow x > 0 \\ 7 & \Leftrightarrow \text{caso contrario} \end{cases}$$

Ejemplos



```
1 int sumaLenta (int a, int b){  
2     while(a > 0){  
3         a= a-1;  
4         b= b+1  
5     }  
6     return b;  
7 }
```


Tiempo de ejecución - Tarea

```
1  int triangulares(int n){
2      int suma;
3      int contador;
4      suma= 0;
5      contador= 1;
6      while(contador <= n){
7          suma= suma + contador;
8          contador= contador + 1;
9      }
10     return suma;
11 }
```

Tiempo de ejecución

UMSS

Todo proceso puede ser *“medido”* mediante una función matemática que determina el *Tiempo de Ejecución* del proceso.

Pero, es necesario encontrar el $T(n)$?, por lo menos a este nivel de detalle?

Pues no, lo único que interesa saber es cuán complejo es el proceso.

Complejidad

0

$$O(g(n)) \equiv \{f(n)/0 \leq f(n) \leq c * g(n), \forall n \geq n_0\}$$

$$f(n) = O(g(n)) \simeq f(n) \leq g(n)$$

 Ω
$$\Omega(g(n)) \equiv \{f(n)/0 \leq c * g(n) \leq f(n), \forall n \geq n_0\}$$

$$f(n) = \Omega(g(n)) \quad \simeq \quad g(n) \leq f(n)$$

 $\ominus$
$$\Theta(g(n)) \equiv \{f(n)/0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n), \forall n \geq n_0\}$$

$$f(n) = \Theta(g(n)) \quad \Leftrightarrow \quad g(n) < f(n) < g(n)$$

Propiedades

► *Transitividad*

- ▶ $f(n) = \Theta(g(n)) \wedge g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$
- ▶ $f(n) = O(g(n)) \wedge g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$
- ▶ $f(n) = \Omega(g(n)) \wedge g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n))$

► *Reflexiva*

- ▶ $f(n) = \Theta(f(n))$
- ▶ $f(n) = O(f(n))$
- ▶ $f(n) = \Omega(f(n))$

► *Simetria*

- $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$

- ▶ *Simetría transpuesta*
 - ▶ $f(n) = O(g(n)) \Rightarrow g(n) = \Omega(f(n))$
- ▶ *Relación orden*
 - ▶ $f(n) = O(g(n)); \quad a \leq b$
 - ▶ $f(n) = \Theta(g(n)); \quad a = b$
 - ▶ $f(n) = \Omega(g(n)); \quad a \geq b$

Para fines de comparación y determinación de eficiencia, usualmente se considera $\odot$, para determinar la optimalidad se utiliza $\ominus$

Cómo determinar la función $g(n)$ dado que se tiene $f(n)$?

Si la función $f(n)$ tiene la forma:

$$f(n) = t_1 + t_2 + t_3 + \dots t_k$$

Se debe identificar el t_j , que sea el mayor de todos los términos de la $f(n)$.

Éste término es el candidato a ser $g(n)$.

Example

Si $f(n) = \frac{9n}{10} + 13$, el término más pesado es: $\frac{9n}{10}$.

Éste término es el más pesado por la presencia de n , por lo que es un factor constante, por lo que al quitar este coeficiente el termino sigue siendo el más pesado - entonces n es nuestro candidato. Por lo tanto, $g(n) = n$

Por el concepto de complejidad: $f(n) = \frac{9n}{10} + 13$, es acotado maximalmente por $g(n) = n$.

Veamos gráficamente las funciones $f(n)$ y $g(n)$

Retomemos: $f(n) = \frac{9n}{10} + 13$, es acotado maximalmente por $g(n) = n$.

Cómo es esto posible?, claramente la función $f(n)$ está más arriba que $g(n)$

Recordemos la definición de O :

$$f(n) = O(g(n)) \simeq f(n) \leq g(n)$$

Que se lee:

$g(n)$ acota maximalmente a $f(n)$

Complejidad

$$f(n) = O(g(n)) \simeq f(n) \leq g(n)$$

Para que esta relación se cumpla, se debe conseguir un n_0 a partir del cual la función $g(n)$ acota maximalmente a $f(n)$. Y a su vez es posible desplazar la función $g(n)$ en un valor constante c . Entonces se tiene que:

$$f(n) = O(g(n))$$

ssi: $\exists c, n_0$, tal que
 $\forall n \geq n_0$, se cumple
 $f(n) \leq c * g(n)$

Complejidad

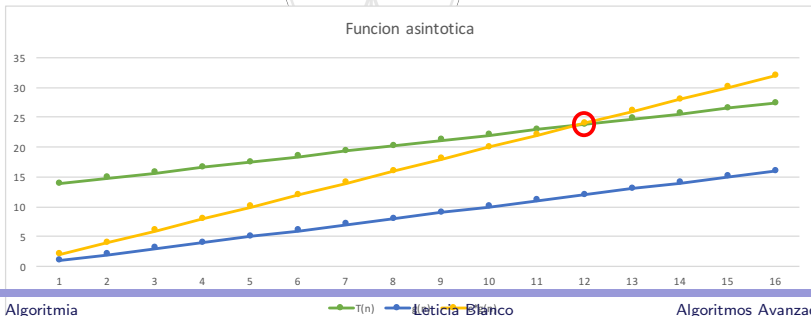
n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
T(n)	13,9	14,8	15,7	16,6	17,5	18,4	19,3	20,2	21,1	22	22,9	23,8	24,7	25,6	26,5	27,4
g(n)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
c*g(n)	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32

$$T(n) = f(n) = 9n/10 + 13$$

$$g(n) = n$$

$$c * g(n) = 2 * n$$

n0 = 12



$$f(n) = O(g(n))$$
$$c = 2$$
$$n_0 = 12$$

Consideremos ahora el $T_{capicua}(x)$ y definamos la función que acota maximalmente a este tiempo de ejecución, para determinar la complejidad de la solución.

$$T_{capicua}(x) = 14 + 16 * \log_{10} x$$

El termino que define la función es

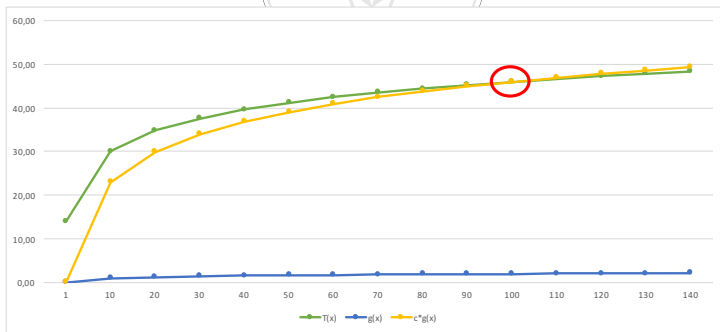
$$16 * \log_{10} x,$$

por lo que la función candidata a acotar es:

$$g(x) = \log_{10} x$$

x	1	10	20	30	40	50	60	70	80	90	100	110	120	130	140
T(x)	14,00	30,00	34,82	37,63	39,63	41,18	42,45	43,52	44,45	45,27	46,00	46,66	47,27	47,82	48,34
g(x)	0,00	1,00	1,30	1,48	1,60	1,70	1,78	1,85	1,90	1,95	2,00	2,04	2,08	2,11	2,15
c*g(x)	0,00	23,00	29,92	33,97	36,85	39,08	40,90	42,44	43,77	44,95	46,00	46,95	47,82	48,62	49,36

$$T(x) = f(x) = 14 + 16 \cdot \log x \quad g(x) = \log x \quad c \cdot g(x) = 23 \cdot \log x \quad n_0 = 100$$



Complejidad

Por lo tanto, se ha demostrado que:

$$14 + 16 * \log x = O(\log n)$$

para valores

$$c = 23$$

$$n_0 = 100$$

La eficiencia se representa con funciones, entonces entenderlas es importante; algunas propiedades son:

► *Monotonicidad*

- ▶ $f(n)$ es monotonicamente incremental ssi
 $m \leq n \Rightarrow f(m) \leq f(n)$
- ▶ $f(n)$ es monotonicamente decremental ssi
 $m \geq n \Rightarrow f(m) \geq f(n)$
- ▶ $f(n)$ es estrictamente incremental ssi
 $m < n \Rightarrow f(m) < f(n)$
- ▶ $f(n)$ es estrictamente decremental ssi
 $m > n \Rightarrow f(m) > f(n)$

$$x \quad 1 \quad \diagdown \quad | \quad \diagup \quad x \quad \diagdown \quad [x] \quad \diagup \quad x \quad | \quad 1$$

$$\forall n \in \mathbb{N} \quad |n/2| \in [n/2] \quad n$$

$$\forall n \in \mathbb{R}^+ \wedge a, b \in \mathbb{Z}^+.$$

$[[r/c]/b]$ $[r/cb]$ Δ Δ

[illegible]

$$\lceil a/b \rceil \leq (a + (b - 1)) / b$$

$$|c/h| \leq (c - (h - 1))/h$$

UMSS

polinomio de grado d , con $d \in \mathbb{Z}^+$

$a_i * n^i$, a_i es coeficiente

$(n^d) \Rightarrow O(n^d)$

1

a

$1/a$

a^{mn}

$(a^n)^m$

a^{m+n}

$$p(n) = \sum_{i=0}^d a_i * n^i, a_i \text{ es coeficiente}$$

$$\Rightarrow p(n) = \Theta(n^d) \Rightarrow O(n^d)$$

$$\forall a \in \mathbb{R}^+$$

$$a^0 = 1$$

$$a^1 = a$$

$$a^{-1} = 1/a$$

$$(a^m)^n = a^{mn}$$

$$(a^m)^n = (a^n)^m$$

$$a^m * a^n = a^{m+n}$$

$$\forall n \wedge a > 1 \Rightarrow a^n \text{ es monotonicamente incremental en } n$$

Funciones

► Logaritmos

$\forall a \in \mathbb{R}, b > 0, c > 0$ y con las bases > 1

$$a = b^{\log_b a}$$

$$\log_c(ab) = \log_c a + \log_c b$$

$$\log_b a^n = n * \log_b a$$

$$\log_b a = \frac{\log_c a}{\log_c b}$$

$$\log_b \frac{1}{a} = -\log_b a$$

$$a^{\log_b c} = c^{\log_b a}$$

$$\frac{x}{1+x} \leq \ln(1+x) \leq x$$

$\forall b > 1 \wedge n > 0 \Rightarrow \log_b n$ es estrictamente incremental en n

Complejidad

Indica si $f(n) = O(g)$ o $f = \Omega(g)$ o si son ambos:

 $f(n)$ $g(n)$

$$\begin{array}{l} n - 100 \\ n^{1/2} \end{array}$$

$$\begin{array}{l} n - 200 \\ n^{2/3} \end{array}$$

$$100n + \log n$$

$$n + (\log n)^2$$

$$n \log n$$

$$10n \log 10n$$

$$\log 2n$$

$$\log 3n$$

$$10 \log n$$

$$\log(n^2)$$

$$n^{1,01}$$

$$n \log^2 n$$

$$n^2 / \log n$$

$$n(\log n)^2$$

$$n^{0,1}$$

$$(\log n)^{10}$$



THOMAS H. CORMEN CHARLES E. LEISERSON RONALD L. RIVEST
CLIFFORD STEIN (2009), *INTRODUCTION TO ALGORITHMS THIRD
EDITION*, The MIT Press